

06_styles-css

#css #html #frontend #responsive #ui #SIEM #SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
├── frontend/  
│   └── assets/  
│       └── styles.css
```

El archivo `styles.css` se encuentra dentro de:

```
frontend/assets/
```

Este archivo define el diseño visual de la interfaz web del laboratorio SIEM MVP.

Su función es aplicar estilos comunes a las páginas:

```
frontend/index.html  
frontend/alert.html
```

A diferencia de los archivos JavaScript, `styles.css` no contiene lógica de conexión con la API. Su papel es visual:

```
HTML  
  ↓  
estructura  
  
JavaScript  
  ↓  
datos y comportamiento  
  
CSS  
  ↓  
presentación visual
```

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,420p' frontend/assets/styles.css
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Permite visualizar contenido de archivos.

```
-n
```

Evita que `sed` imprima todo automáticamente.

```
'1,420p'
```

Imprime desde la línea 1 hasta la 420.

Ruta del archivo analizado.

3 Código completo del archivo

```
:root {
  --bg: #0b1020;
  --card: #111836;
  --card2: #0f1530;
  --text: #e7ecff;
  --muted: #a7b0d6;
  --border: rgba(255,255,255,0.12);
  --border2: rgba(255,255,255,0.08);
  --shadow: 0 10px 30px rgba(0,0,0,0.35);
  --radius: 14px;
}

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: ui-sans-serif, system-ui, -apple-system, Segoe UI, Roboto, Arial, "Noto Sans",
"Liberation Sans", sans-serif;
  background: radial-gradient(1200px 600px at 20% 0%, #17235a 0%, var(--bg) 55%);
  color: var(--text);
}

.container { max-width: 1200px; margin: 0 auto; padding: 16px; }

.topbar {
  position: sticky; top: 0;
  backdrop-filter: blur(10px);
  background: rgba(11,16,32,0.7);
  border-bottom: 1px solid var(--border2);
  z-index: 10;
}

.topbar .container {
  display: flex; align-items: center; justify-content: space-between;
  gap: 16px;
}

.brand { display: flex; align-items: center; gap: 12px; }
.logo {
  display: inline-flex; align-items: center; justify-content: center;
  width: 44px; height: 44px;
  border-radius: 12px;
  background: linear-gradient(135deg, #5a7dff, #9b5cff);
  box-shadow: var(--shadow);
  font-weight: 800;
}

h1 { font-size: 18px; margin: 0; }
h2 { font-size: 16px; margin: 0; }
h3 { font-size: 14px; margin: 16px 0 8px; }
p { margin: 6px 0 0; }
.muted { color: var(--muted); }
.mono { font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace; }

.card {
  background: linear-gradient(180deg, var(--card), var(--card2));
  border: 1px solid var(--border2);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}
```

```

margin: 16px 0;
overflow: hidden;
}
.card-header { padding: 14px 16px; border-bottom: 1px solid var(--border2); }
.card-footer { padding: 10px 16px; border-top: 1px solid var(--border2); }
.row { display: flex; align-items: center; justify-content: space-between; gap: 12px; flex-wrap: wrap; }

.filters {
padding: 14px 16px;
display: grid;
grid-template-columns: repeat(4, minmax(160px, 1fr));
gap: 12px;
}
.field { display: flex; flex-direction: column; gap: 6px; }
.field label { font-size: 12px; color: var(--muted); }
.field input, .field select {
border: 1px solid var(--border);
background: rgba(255,255,255,0.06);
color: var(--text);
padding: 10px 10px;
border-radius: 10px;
outline: none;
}
.field input:focus, .field select:focus { border-color: rgba(90,125,255,0.7); }
.field.buttons { display: flex; gap: 10px; align-items: end; flex-direction: row; }

.btn {
border: 1px solid var(--border);
background: rgba(255,255,255,0.06);
color: var(--text);
padding: 10px 12px;
border-radius: 10px;
cursor: pointer;
text-decoration: none;
display: inline-flex;
gap: 8px;
align-items: center;
justify-content: center;
}
.btn:hover { border-color: rgba(255,255,255,0.22); }
.btn.primary { background: rgba(90,125,255,0.25); border-color: rgba(90,125,255,0.45); }
.btn.danger { background: rgba(255,92,92,0.18); border-color: rgba(255,92,92,0.35); }
.btn.disabled { opacity: 0.55; cursor: not-allowed; }

.pager { display: flex; gap: 10px; }

.alert {
margin: 12px 16px 0;
padding: 10px 12px;
border-radius: 10px;
border: 1px solid var(--border);
}
.alert.error { background: rgba(255,92,92,0.12); border-color: rgba(255,92,92,0.28); }
.alert.info { background: rgba(90,125,255,0.12); border-color: rgba(90,125,255,0.28); }
.hidden { display: none; }

.table-wrap { overflow: auto; }
.table {
width: 100%;
border-collapse: collapse;
min-width: 900px;
}
.table th, .table td {
padding: 10px 12px;
border-bottom: 1px solid var(--border2);

```

```

text-align: left;
vertical-align: top;
}
.table th { font-size: 12px; color: var(--muted); font-weight: 600; }
.table tr:hover td { background: rgba(255,255,255,0.03); }

.badge {
  display: inline-flex;
  padding: 2px 8px;
  border-radius: 999px;
  border: 1px solid var(--border);
  font-size: 12px;
  text-transform: lowercase;
}

.grid {
  padding: 14px 16px;
  display: grid;
  grid-template-columns: minmax(260px, 1fr) minmax(260px, 1fr);
  gap: 14px;
}

.kv {
  display: grid;
  grid-template-columns: 140px 1fr;
  gap: 8px 12px;
  align-content: start;
  padding: 12px;
  border: 1px solid var(--border2);
  border-radius: 12px;
  background: rgba(255,255,255,0.03);
}

.k { color: var(--muted); font-size: 12px; }
.v { font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace; }

.panel {
  padding: 12px;
  border: 1px solid var(--border2);
  border-radius: 12px;
  background: rgba(255,255,255,0.03);
}

@media (max-width: 980px) {
  .filters { grid-template-columns: repeat(2, minmax(160px, 1fr)); }
  .grid { grid-template-columns: 1fr; }
}

```

4 Función general del archivo

styles.css define la apariencia visual del frontend.

Este archivo controla:

- Colores globales.
- Tipografía.
- Fondo de la aplicación.
- Cabecera superior.
- Contenedores.
- Tarjetas.
- Formularios.
- Botones.
- Mensajes de error e información.
- Tablas.
- Badges de estado.

- Página de detalle.
- Diseño responsive.

La relación con el resto del frontend es:

```
index.html
  ↓
usa clases como topbar, container, card, filters, table, btn

alert.html
  ↓
usa clases como topbar, card, grid, kv, panel, badge

styles.css
  ↓
define cómo se ven esas clases
```

Sin este archivo, el frontend seguiría mostrando datos, pero con aspecto básico de navegador.

5 Estructura general del archivo

El archivo puede dividirse en estos bloques:

1. Variables globales CSS.
2. Reset básico y body.
3. Contenedor general.
4. Topbar, brand y logo.
5. Tipografía y utilidades.
6. Cards.
7. Formularios.
8. Botones.
9. Alertas visuales.
10. Tablas.
11. Badges.
12. Grid de detalle.
13. Paneles.
14. Media query responsive.

Visualmente:

```
styles.css
├─ :root
├─ reset básico
├─ body
├─ container
├─ topbar
├─ brand / logo
├─ títulos / utilidades
├─ card / card-header / card-footer
├─ filters / field
├─ btn
├─ alert / hidden
├─ table
├─ badge
├─ grid / kv
├─ panel
└─ media query
```

6 Análisis línea por línea

Variables globales

```
:root {
```

:root representa el elemento raíz del documento.

En HTML, equivale normalmente a:

```
html
```

Se utiliza para definir variables CSS globales.

Estas variables después se reutilizan con:

```
var(--nombre)
```

Variable --bg

```
--bg: #0b1020;
```

Define el color de fondo principal.

Es un azul oscuro.

Se usa en el `body` como parte del fondo:

```
var(--bg)
```

Variable --card

```
--card: #111836;
```

Define el color principal de las tarjetas.

Se usa en:

```
.card
```

dentro del gradiente de fondo.

Variable --card2

```
--card2: #0f1530;
```

Define el segundo color del gradiente de las tarjetas.

Esto permite que las tarjetas no sean completamente planas.

Variable --text

```
--text: #e7ecff;
```

Define el color principal del texto.

Se aplica al `body` :

```
color: var(--text);
```

Variable `--muted`

```
--muted: #a7b0d6;
```

Define el color para textos secundarios.

Se usa en la clase:

```
.muted
```

Ejemplos de uso:

```
<p class="muted">Vista principal – listado, filtros y paginación</p>
```

Variable `--border`

```
--border: rgba(255,255,255,0.12);
```

Define un borde claro con transparencia.

Se usa en botones, inputs, alertas y badges.

Variable `--border2`

```
--border2: rgba(255,255,255,0.08);
```

Define un borde todavía más sutil.

Se usa en tarjetas, tablas y separadores.

Variable `--shadow`

```
--shadow: 0 10px 30px rgba(0,0,0,0.35);
```

Define una sombra reutilizable.

Se usa en:

```
.logo  
.card
```

La sombra ayuda a separar visualmente elementos del fondo.

Variable `--radius`

```
--radius: 14px;
```

Define el redondeo estándar.

Se usa en:

```
.card
```

Esto mantiene una estética consistente.

Cierre de `:root`

```
}
```

Cierra el bloque de variables globales.

Reset de box model

```
* { box-sizing: border-box; }
```

Aplica `box-sizing: border-box` a todos los elementos.

Esto hace que el ancho total de un elemento incluya:

```
contenido  
padding  
borde
```

Es una práctica habitual para evitar cálculos incómodos de anchura.

Sin esto, un elemento con `width: 100%` y padding podría desbordarse.

Estilo general del `body`

```
body {
```

Define los estilos globales del cuerpo del documento.

Todo lo visible en la página está dentro de `body`.

Eliminar margen por defecto

```
margin: 0;
```

Elimina el margen que los navegadores aplican por defecto al `body`.

Esto permite controlar el espaciado desde cero.

Fuente principal

```
font-family: ui-sans-serif, system-ui, -apple-system, Segoe UI, Roboto, Arial, "Noto Sans",  
"Liberation Sans", sans-serif;
```

Define la familia tipográfica del frontend.

Usa una pila de fuentes del sistema.

Ventajas:

- Carga rápida.
 - Buena integración con el sistema operativo.
 - No depende de fuentes externas.
-

Fondo visual

```
background: radial-gradient(1200px 600px at 20% 0%, #17235a 0%, var(--bg) 55%);
```

Define el fondo de la página.

Usa un gradiente radial.

Desglose:

```
radial-gradient(...)
```

Crea un degradado circular o elíptico.

```
1200px 600px
```

Define el tamaño del gradiente.

```
at 20% 0%
```

Sitúa el centro del gradiente cerca de la zona superior izquierda.

```
#17235a 0%
```

Color inicial del gradiente.

```
var(--bg) 55%
```

Color final a partir del 55%.

Esto da un efecto visual oscuro con iluminación superior.

Color de texto global

```
color: var(--text);
```

Aplica el color principal del texto definido en `:root`.

Cierre de `body`

```
}
```

Cierra los estilos globales del cuerpo.

Contenedor principal

```
.container { max-width: 1200px; margin: 0 auto; padding: 16px; }
```

Define una clase reutilizable para centrar contenido.

Desglose:

```
max-width: 1200px;
```

Limita el ancho máximo del contenido.

```
margin: 0 auto;
```

Centra horizontalmente el contenedor.

```
padding: 16px;
```

Añade espacio interno.

Esta clase se usa en:

```
<header class="topbar">
  <div class="container">
```

y:

```
<main class="container">
```

Cabecera superior

```
.topbar {
```

Define el estilo de la barra superior.

Se usa en:

```
<header class="topbar">
```

Posición sticky

```
position: sticky; top: 0;
```

Hace que la cabecera quede pegada arriba cuando se hace scroll.

Desglose:

```
position: sticky;
```

Permite que el elemento se comporte como relativo hasta llegar a una posición límite.

```
top: 0;
```

La posición límite es la parte superior de la ventana.

Efecto blur

```
backdrop-filter: blur(10px);
```

Aplica desenfoque al contenido que queda detrás de la cabecera.

Esto crea un efecto visual tipo cristal.

Fondo translúcido

```
background: rgba(11,16,32,0.7);
```

Define un fondo oscuro con transparencia.

El último valor `0.7` representa la opacidad.

Borde inferior

```
border-bottom: 1px solid var(--border2);
```

Añade una línea inferior sutil.

Esto separa la cabecera del contenido.

Nivel de apilamiento

```
z-index: 10;
```

Hace que la cabecera quede por encima de otros elementos.

Es importante porque usa `position: sticky`.

Contenedor dentro de topbar

```
.topbar .container {  
  display: flex; align-items: center; justify-content: space-between;  
  gap: 16px;  
}
```

Aplica estilos al `.container` cuando está dentro de `.topbar`.

Flexbox

```
display: flex;
```

Organiza los elementos hijos en una fila.

Alineación vertical

```
align-items: center;
```

Centra los elementos verticalmente.

Separación horizontal

```
justify-content: space-between;
```

Coloca el bloque de marca a la izquierda y las acciones a la derecha.

Espacio entre elementos

```
gap: 16px;
```

Añade separación entre elementos hijos.

Marca

```
.brand { display: flex; align-items: center; gap: 12px; }
```

Define el bloque del logo y los textos principales.

Se usa en:

```
<div class="brand">
```

display: flex

Organiza logo y texto en horizontal.

`align-items: center`

Centra verticalmente el logo y los textos.

`gap: 12px`

Añade espacio entre logo y título.

Logo

```
.logo {
```

Define el aspecto del elemento:

```
<span class="logo">SIEM</span>
```

Layout interno del logo

```
display: inline-flex; align-items: center; justify-content: center;
```

Hace que el contenido del logo quede centrado tanto vertical como horizontalmente.

Tamaño del logo

```
width: 44px; height: 44px;
```

Define un recuadro de 44x44 píxeles.

Redondeo del logo

```
border-radius: 12px;
```

Redondea las esquinas.

Fondo del logo

```
background: linear-gradient(135deg, #5a7dff, #9b5cff);
```

Aplica un gradiente diagonal.

Sombra del logo

```
box-shadow: var(--shadow);
```

Aplica la sombra definida en `:root`.

Peso de fuente

```
font-weight: 800;
```

Hace que el texto `SIEM` aparezca en negrita.

Cierre del logo

```
}
```

Cierra el bloque `.logo`.

Títulos

```
h1 { font-size: 18px; margin: 0; }  
h2 { font-size: 16px; margin: 0; }  
h3 { font-size: 14px; margin: 16px 0 8px; }
```

Define tamaños y márgenes para títulos.

h1

Título principal de la página.

Ejemplos:

```
Alertas  
Detalle de alerta
```

h2

Título de sección.

Ejemplos:

```
Filtros  
Listado  
Alerta #—
```

h3

Subtítulos internos.

Ejemplo:

```
Título  
Notas
```

Tiene margen superior para separarlo del contenido anterior.

Párrafos

```
p { margin: 6px 0 0; }
```

Define un margen superior ligero para párrafos.

Clase `muted`

```
.muted { color: var(--muted); }
```

Aplica color secundario.

Se usa en textos informativos o menos importantes.

Ejemplos:

```
<p class="muted">Vista principal – listado, filtros y paginación</p>
```

Clase mono

```
.mono { font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace; }
```

Aplica una fuente monoespaciada.

Se usa para IDs, fechas y datos técnicos.

Esto mejora la legibilidad de valores como:

```
rule_id  
event_id  
created_at
```

Tarjetas

```
.card {
```

Define el estilo visual de las tarjetas principales.

Se usa en:

```
<section class="card">
```

Fondo de tarjeta

```
background: linear-gradient(180deg, var(--card), var(--card2));
```

Aplica un gradiente vertical usando las variables `--card` y `--card2`.

Borde de tarjeta

```
border: 1px solid var(--border2);
```

Añade un borde sutil.

Redondeo de tarjeta

```
border-radius: var(--radius);
```

Usa el radio global definido en `:root`.

Sombra de tarjeta

```
box-shadow: var(--shadow);
```

Aplica la sombra global.

Esto separa visualmente la tarjeta del fondo.

Margen de tarjeta

```
margin: 16px 0;
```

Añade separación vertical entre tarjetas.

Recorte interno

```
overflow: hidden;
```

Impide que contenido interno sobresalga de los bordes redondeados.

Cabecera de tarjeta

```
.card-header { padding: 14px 16px; border-bottom: 1px solid var(--border2); }
```

Define la cabecera interna de las tarjetas.

Añade:

```
padding
borde inferior
```

Pie de tarjeta

```
.card-footer { padding: 10px 16px; border-top: 1px solid var(--border2); }
```

Define el pie de tarjeta.

Se usa en `index.html` para la nota de severidad.

Clase `row`

```
.row { display: flex; align-items: center; justify-content: space-between; gap: 12px; flex-wrap: wrap; }
```

Permite colocar elementos en fila.

Se usa en cabeceras donde hay texto a un lado y botones al otro.

Desglose:

```
display: flex;
```

Activa flexbox.

```
align-items: center;
```

Centra verticalmente.

```
justify-content: space-between;
```

Separa los extremos.

```
gap: 12px;
```

Añade espacio.

```
flex-wrap: wrap;
```

Permite que los elementos bajen de línea si no caben.

Esto mejora la adaptación a pantallas pequeñas.

Formulario de filtros

```
.filters {
```

Define el layout del formulario de filtros de `index.html`.

Padding de filtros

```
padding: 14px 16px;
```

Añade espacio interno.

Grid de filtros

```
display: grid;
grid-template-columns: repeat(4, minmax(160px, 1fr));
gap: 12px;
```

Organiza los filtros en una cuadrícula.

Desglose:

```
display: grid;
```

Activa CSS Grid.

```
grid-template-columns: repeat(4, minmax(160px, 1fr));
```

Crea 4 columnas.

Cada columna tiene mínimo 160px y puede crecer hasta ocupar espacio disponible.

```
gap: 12px;
```

Espacio entre campos.

Campo de formulario

```
.field { display: flex; flex-direction: column; gap: 6px; }
```

Cada campo se organiza en columna:

```
label
input/select
```

Label de campo


```
.field label { font-size: 12px; color: var(--muted); }
```

Hace que las etiquetas sean pequeñas y secundarias.

Inputs y selects

```
.field input, .field select {
```

Aplica estilos comunes a campos de texto y selectores.

Borde de input/select

```
border: 1px solid var(--border);
```

Añade borde visible.

Fondo de input/select

```
background: rgba(255,255,255,0.06);
```

Fondo claro muy transparente.

Color de texto

```
color: var(--text);
```

Usa el color principal.

Padding

```
padding: 10px 10px;
```

Añade espacio interno.

Redondeo

```
border-radius: 10px;
```

Redondea esquinas.

Eliminar outline por defecto

```
outline: none;
```

Elimina el contorno por defecto del navegador.

El foco se gestiona con una regla específica.

Foco en inputs/selects

```
.field input:focus, .field select:focus { border-color: rgba(90,125,255,0.7); }
```

Cuando un campo está enfocado, cambia el color del borde.

Esto ayuda a saber qué campo está activo.

Campo de botones

```
.field.buttons { display: flex; gap: 10px; align-items: end; flex-direction: row; }
```

Hace que los botones del formulario estén en fila.

Desglose:

```
display: flex;
```

Activa flexbox.

```
gap: 10px;
```

Separa botones.

```
align-items: end;
```

Alinea los botones hacia abajo.

```
flex-direction: row;
```

Coloca botones horizontalmente.

Botones

```
.btn {
```

Define el estilo base de todos los botones y enlaces con aspecto de botón.

Se usa en:

```
<button class="btn">  
<a class="btn">
```

Borde del botón

```
border: 1px solid var(--border);
```

Borde sutil.

Fondo del botón

```
background: rgba(255,255,255,0.06);
```

Fondo semitransparente.

Color del botón

```
color: var(--text);
```

Texto claro.

Padding del botón

```
padding: 10px 12px;
```

Espacio interno.

Redondeo del botón

```
border-radius: 10px;
```

Redondea esquinas.

Cursor

```
cursor: pointer;
```

Muestra cursor de clic.

Quitar subrayado en enlaces

```
text-decoration: none;
```

Importante porque algunos botones son enlaces `<a>`.

Layout interno del botón

```
display: inline-flex;  
gap: 8px;  
align-items: center;  
justify-content: center;
```

Permite alinear texto e iconos si los hubiera.

Hover del botón

```
.btn:hover { border-color: rgba(255,255,255,0.22); }
```

Al pasar el cursor, el borde se aclara.

Esto da feedback visual.

Botón principal

```
.btn.primary { background: rgba(90,125,255,0.25); border-color: rgba(90,125,255,0.45); }
```

Define el estilo para acciones principales.

Se usa en:

```
<button class="btn primary">Aplicar</button>  
<button id="btnAck" class="btn primary">ACK</button>
```

Botón peligro

```
.btn.danger { background: rgba(255,92,92,0.18); border-color: rgba(255,92,92,0.35); }
```

Define el estilo para acciones más críticas.

Se usa en:

```
<button id="btnClose" class="btn danger">CLOSE</button>
```

Botón deshabilitado

```
.btn:disabled { opacity: 0.55; cursor: not-allowed; }
```

Cuando un botón está deshabilitado:

- se ve más transparente
- muestra cursor de no permitido

Esto se usa en la lógica de detalle de alerta.

Paginador

```
.pager { display: flex; gap: 10px; }
```

Coloca los botones de paginación en fila con separación.

Se usa en `index.html` para:

```
Prev  
Next
```

Alertas visuales

```
.alert {
```

Define las cajas de mensajes de error e información.

No son alertas SIEM. Son mensajes de interfaz.

Margen de alerta visual

```
margin: 12px 16px 0;
```

Añade separación superior y lateral.

Padding de alerta visual

```
padding: 10px 12px;
```

Espacio interno.

Redondeo de alerta visual

```
border-radius: 10px;
```

Redondea esquinas.

Borde de alerta visual

```
border: 1px solid var(--border);
```

Borde base.

Alerta de error

```
.alert.error { background: rgba(255,92,92,0.12); border-color: rgba(255,92,92,0.28); }
```

Define el aspecto de errores.

Se usa en:

```
<div id="errorBox" class="alert error hidden"></div>
```

Alerta de información

```
.alert.info { background: rgba(90,125,255,0.12); border-color: rgba(90,125,255,0.28); }
```

Define el aspecto de mensajes informativos.

Se usa en:

```
<div id="infoBox" class="alert info hidden"></div>
```

Ocultar elementos

```
.hidden { display: none; }
```

Ocultar cualquier elemento con la clase `hidden`.

JavaScript muestra u oculta elementos añadiendo o quitando esta clase.

Ejemplo:

```
el.classList.remove("hidden");  
el.classList.add("hidden");
```

Contenedor de tabla

```
.table-wrap { overflow: auto; }
```

Permite scroll si la tabla es más ancha que la pantalla.

Esto es importante porque la tabla tiene muchas columnas.

Tabla

```
.table {  
  width: 100%;
```

```
border-collapse: collapse;
min-width: 900px;
}
```

Define la tabla principal.

Ancho completo

```
width: 100%;
```

La tabla intenta ocupar todo el ancho del contenedor.

Colapsar bordes

```
border-collapse: collapse;
```

Hace que los bordes de celdas se fusionen.

Ancho mínimo

```
min-width: 900px;
```

Evita que la tabla se comprima demasiado.

Si la pantalla es estrecha, entra en juego `.table-wrap` con scroll horizontal.

Celdas de tabla

```
.table th, .table td {
  padding: 10px 12px;
  border-bottom: 1px solid var(--border2);
  text-align: left;
  vertical-align: top;
}
```

Aplica estilos a cabeceras y celdas.

Padding de celdas

```
padding: 10px 12px;
```

Hace que la tabla respire visualmente.

Borde inferior

```
border-bottom: 1px solid var(--border2);
```

Separa filas.

Alineación horizontal

```
text-align: left;
```

Alinea texto a la izquierda.

Alineación vertical

```
vertical-align: top;
```

Alinea contenido arriba.

Útil si algún texto ocupa varias líneas.

Cabeceras de tabla

```
.table th { font-size: 12px; color: var(--muted); font-weight: 600; }
```

Define el aspecto de las cabeceras.

Son más pequeñas y con color secundario.

Hover de fila

```
.table tr:hover td { background: rgba(255,255,255,0.03); }
```

Cuando el usuario pasa el ratón por una fila, las celdas cambian ligeramente de fondo.

Esto ayuda a leer tablas.

Badge

```
.badge {
```

Define el estilo visual de estados como:

```
open
ack
closed
```

Layout del badge

```
display: inline-flex;
```

Permite que el badge se comporte como una caja en línea.

Padding del badge

```
padding: 2px 8px;
```

Espacio interno pequeño.

Forma redondeada

```
border-radius: 999px;
```

Crea forma tipo píldora.

Borde del badge

```
border: 1px solid var(--border);
```

Borde sutil.

Tamaño de fuente

```
font-size: 12px;
```

Texto pequeño.

Texto en minúsculas

```
text-transform: lowercase;
```

Fuerza el texto a minúsculas.

Aunque llegara `OPEN` , se mostraría como `open` .

Grid de detalle

```
.grid {
```

Define el layout de la página de detalle.

Se usa en `alert.html` .

Padding del grid

```
padding: 14px 16px;
```

Espacio interno.

Activar grid

```
display: grid;
```

Activa CSS Grid.

Dos columnas

```
grid-template-columns: minmax(260px, 1fr) minmax(260px, 1fr);
```

Crea dos columnas flexibles.

Cada una tiene mínimo 260px y puede crecer.

Esto permite colocar:

```
datos clave-valor  
panel de título/notas
```

en paralelo.

Espacio entre columnas


```
gap: 14px;
```

Separa los bloques.

Bloque key-value

```
.kv {
```

Define el bloque de datos técnicos de `alert.html`.

Grid interno de key-value

```
display: grid;  
grid-template-columns: 140px 1fr;
```

Crea dos columnas:

```
clave → 140px  
valor → resto
```

Espaciado interno

```
gap: 8px 12px;
```

Define espacio entre filas y columnas.

Alineación de contenido

```
align-content: start;
```

Alinea el contenido al principio del bloque.

Padding del bloque

```
padding: 12px;
```

Espacio interno.

Borde del bloque

```
border: 1px solid var(--border2);
```

Borde sutil.

Redondeo del bloque

```
border-radius: 12px;
```

Esquinas redondeadas.

Fondo del bloque

```
background: rgba(255,255,255,0.03);
```

Fondo ligeramente diferenciado.

Clave `.k`

```
.k { color: var(--muted); font-size: 12px; }
```

Define las etiquetas del bloque key-value.

Ejemplos:

```
id
status
group_key
rule_id
event_id
```

Se muestran pequeñas y secundarias.

Valor `.v`

```
.v { font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace; }
```

Los valores se muestran con fuente monoespaciada.

Esto mejora lectura de IDs, fechas y claves técnicas.

Panel

```
.panel {
```

Define el panel lateral de la página de detalle.

Padding del panel

```
padding: 12px;
```

Espacio interno.

Borde del panel

```
border: 1px solid var(--border2);
```

Borde sutil.

Redondeo del panel

```
border-radius: 12px;
```

Esquinas redondeadas.

Fondo del panel

```
background: rgba(255,255,255,0.03);
```

Fondo ligeramente diferenciado.

Media query responsive

```
@media (max-width: 980px) {
```

Define reglas que solo se aplican cuando la pantalla tiene ancho máximo de 980px.

Sirve para adaptar el diseño a pantallas más pequeñas.

Filtros en pantallas pequeñas

```
.filters { grid-template-columns: repeat(2, minmax(160px, 1fr)); }
```

Cuando la pantalla es menor de 980px, los filtros pasan de 4 columnas a 2.

Antes:

```
4 columns
```

Después:

```
2 columns
```

Esto evita que los campos queden demasiado estrechos.

Grid de detalle en una columna

```
.grid { grid-template-columns: 1fr; }
```

En pantallas pequeñas, el detalle de alerta pasa de dos columnas a una.

Antes:

```
kv | panel
```

Después:

```
kv  
panel
```

Esto mejora la lectura en pantallas estrechas.

Cierre de media query

```
}
```

Cierra el bloque responsive.

7 Relación con el flujo técnico del laboratorio

`styles.css` no participa en la lógica de backend ni en las llamadas API.

Su papel es presentar visualmente la información que ya viene de:

FastAPI
PostgreSQL
JavaScript
HTML

Relación general:

```
Backend
  ↓
API JSON
  ↓
JavaScript
  ↓
HTML dinámico
  ↓
CSS
  ↓
Interfaz visual usable
```

En el flujo del laboratorio:

```
GET /alerts
  ↓
alerts.js
  ↓
index.html
  ↓
styles.css
  ↓
tabla visual de alertas
```

Y en detalle:

```
GET /alerts/{id}
  ↓
alert_detail.js
  ↓
alert.html
  ↓
styles.css
  ↓
vista visual de detalle
```

8 Relación entre CSS y HTML

Las clases definidas en `styles.css` se usan directamente en los archivos HTML.

Ejemplos:

<code>.topbar</code>	→ cabecera superior
<code>.container</code>	→ ancho máximo y centrado
<code>.brand</code>	→ logo + texto
<code>.logo</code>	→ recuadro SIEM
<code>.card</code>	→ tarjetas de contenido
<code>.card-header</code>	→ cabecera de tarjeta
<code>.filters</code>	→ formulario de filtros
<code>.field</code>	→ campo individual
<code>.btn</code>	→ botones y enlaces
<code>.table</code>	→ tabla de alertas
<code>.badge</code>	→ estado de alerta
<code>.grid</code>	→ layout de detalle
<code>.kv</code>	→ bloque clave-valor

<code>.panel</code>	→ panel lateral
<code>.hidden</code>	→ ocultar mensajes

La relación es:

```
HTML class="btn"
  ↓
CSS .btn
  ↓
botón con estilo
```

9 Errores típicos o puntos importantes

Las variables CSS centralizan el diseño

Colores, bordes, sombras y radios están definidos en `:root`.

Esto facilita cambios globales.

Si se cambia:

```
--bg
```

cambia el fondo principal.

Si se cambia:

```
--radius
```

cambia el redondeo estándar.

`.hidden` depende de JavaScript

La clase:

```
.hidden { display: none; }
```

se usa desde JavaScript para mostrar u ocultar mensajes.

Funciones relacionadas:

```
show()
hide()
```

`.table-wrap` evita romper el diseño

La tabla tiene:

```
min-width: 900px;
```

En pantallas pequeñas, podría desbordar.

El contenedor:

```
.table-wrap { overflow: auto; }
```

permite scroll horizontal.

Los botones deshabilitados se entienden visualmente

La regla:

```
.btn:disabled
```

hace que un botón deshabilitado parezca realmente no disponible.

Esto es importante en `alert.html` , donde ACK/CLOSE/REOPEN cambian según estado.

El diseño responsive es básico pero suficiente

Solo hay una media query:

```
@media (max-width: 980px)
```

Con eso se adaptan filtros y detalle.

Para un MVP es suficiente.

.badge no diferencia estados por color

Actualmente todos los estados usan la misma clase visual:

```
.badge
```

Esto coincide con `app.js` , donde:

```
statusBadgeClass()
```

devuelve siempre `"badge"` .

Una mejora futura sería añadir clases por estado.

10 Posibles mejoras futuras

Diferenciar badges por estado

Se podrían añadir estilos como:

```
.badge.open { ... }  
.badge.ack { ... }  
.badge.closed { ... }
```

Y modificar `statusBadgeClass()` para devolver:

```
return `badge ${status}`;
```

Así cada estado tendría una lectura visual más rápida.

Añadir modo claro

Como los colores están centralizados en `:root` , sería posible crear un tema claro.

Ejemplo:

```
body.light {  
  --bg: #f5f7fb;  
  --card: #ffffff;  
  --text: #111827;  
}
```

Mejorar responsive móvil

Actualmente los filtros pasan a dos columnas.

En móviles muy pequeños podrían pasar a una columna:

```
@media (max-width: 600px) {  
  .filters { grid-template-columns: 1fr; }  
}
```

Añadir estilos para severidad

Si el frontend pasa a consumir `/alerts/ui`, podría mostrar severidades con clases visuales.

Ejemplo:

```
.severity-high  
.severity-medium  
.severity-low
```

Añadir estados visuales de carga

Ahora se muestra texto `Cargando...`.

Podría añadirse un spinner o skeleton simple.

1 1 Comandos útiles relacionados

Servir frontend:

```
cd ~/siem-lab/frontend  
python3 -m http.server 5173
```

Abrir página principal:

```
http://localhost:5173/index.html
```

Abrir detalle de alerta:

```
http://localhost:5173/alert.html?id=1
```

Abrir herramientas de desarrollo del navegador:

```
F12 → Elements
```

Comprobar CSS cargado:

```
Network → styles.css
```

Probar cambios rápidos:

```
F12 → Elements → seleccionar elemento → modificar reglas CSS
```

1 2 Resumen técnico

`styles.css` define la capa visual del frontend del laboratorio SIEM MVP.

El archivo utiliza variables CSS globales para mantener consistencia en colores, bordes, sombras y radios. Define una interfaz oscura, con tarjetas, botones, tablas, badges y paneles de detalle.

Su papel dentro del proyecto es convertir la información técnica de alertas en una interfaz más clara y usable.

La relación final es:

```
HTML
↓
estructura

JavaScript
↓
datos y comportamiento

CSS
↓
presentación visual
```

Con esta nota queda cerrado el módulo:

```
07_Frontend
└─ 08_Analisis-tecnico-frontend
   └─ 01_index-html
   └─ 02_alert-html
   └─ 03_app-js
   └─ 04_alerts-js
   └─ 05_alert-detail-js
   └─ 06_styles-css
```